

Travel and Tourism

BlazeDemo Flight Booking Automation Framework

Selenium Automation Capstone Project Report

Name: **Sumana Chowdhury**

Batch: **SA2510014**

Role: **Automation Trainee**

Technology: **Selenium WebDriver with Java**

Framework: **TestNG + Maven**

Design Pattern: **Page Object Model (POM) with Page Factory**

Automation Techniques: **Data Driven Testing, Screenshot Capture, Extent Reports**

Table of Contents :

1. Introduction
2. Project Objective
3. Tools and Technologies Used
4. Framework Architecture
5. Project Folder Structure
6. Test Scenarios
7. Page Object Model with Page Factory Implementation
8. Data Driven Testing Implementation
9. Screenshot Utility Implementation
10. Test Execution Process
11. Console Execution Screenshot
12. Extent Report Screenshot
13. Challenges Faced
14. Test Script Screenshots
15. Conclusion

1. Introduction

BlazeDemo is a flight booking demo website designed to mimic a real-world flight booking platform used for automation practice.

The objective of this project is to automate the end-to-end flight booking workflow using Selenium WebDriver with Java.

The Framework is designed using **Page Object Model with Page Factory**, enabling better code maintainability and reusability.

2. Project Objective

The objective of this automation project is :

- Automate flight booking workflow
- Implement Page Object Model framework
- Use Page Factory for element initialization
- Implement Data Driven Testing
- Execute test using testing
- Validate booking confirmation
- Generate execution reports
- Capture Screenshots

3. Tools and Technologies Used

Tools	Purpose
Selenium WebDriver	Browser automation
Java	Programming Language
TestNG	Test Execution
Maven	Dependency Management
Extent Reports	Reporting
Eclipse IDE	Development Environment

4. Framework Architecture

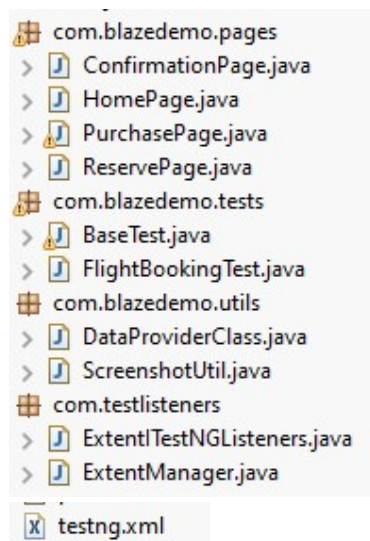
The automation framework for the BlazeDemo application is designed using the **Page Object Model (POM) with Page Factory** designed pattern.

Page Object Model improves the maintainability and scalability of the test automation framework by separating **page elements, page actions and test scripts** into different classes.

Each web page of the BlazeDemo application is represented by a separate Java class which contains:

- Web element locators
- Page actions (Methods that perform operations on elements)
- Page initialization using Page Factory

5. Project Folder Structure



Pages Packages contains all page classes and web elements.

Tests Package contains automation test scripts.

Utils Package contains data provider and utility classes.

Testlisteners Package contains TestNG listener classes used for reporting and logging.

6. Test Scenarios

Positive test case scenarios

Test Case ID	Description	Expected Result	Type
TC01	Launch browser	Browser should open	Smoke
TC02	verify homepage and dropdowns	page should and dropdowns visible	Smoke
TC03	Search flights with valid cities	Flight search page open	Functional
TC04	Select flight	Purchase page opens	Functional
TC05	Multiple bookings with different data sets	Booking confirmed	Data Driven

7. Page Object Model with Page Factory Implementation

Page Object Model is a design pattern used to separate web elements and test logic.

Benefits of POM :

- ✓ Better code maintenance
- ✓ Improved readability
- ✓ Reusable test scripts

Example code :

```
//Locators
//Departure City Dropdown
@FindBy(name="fromPort")
private WebElement depCity;

// Destination City DropDown
@FindBy(name="toPort")
private WebElement desCity;

// Find Flight Button
@FindBy(xpath="//input[@value=\"Find Flights\"]")
WebElement findFlights;
```

Page Factory initializes elements using

```
PageFactory.initElements(driver,this);
```

8. Data Driven Testing Implementation

Data driven Testing allows running test cases with multiple data inputs.

Example code :

```

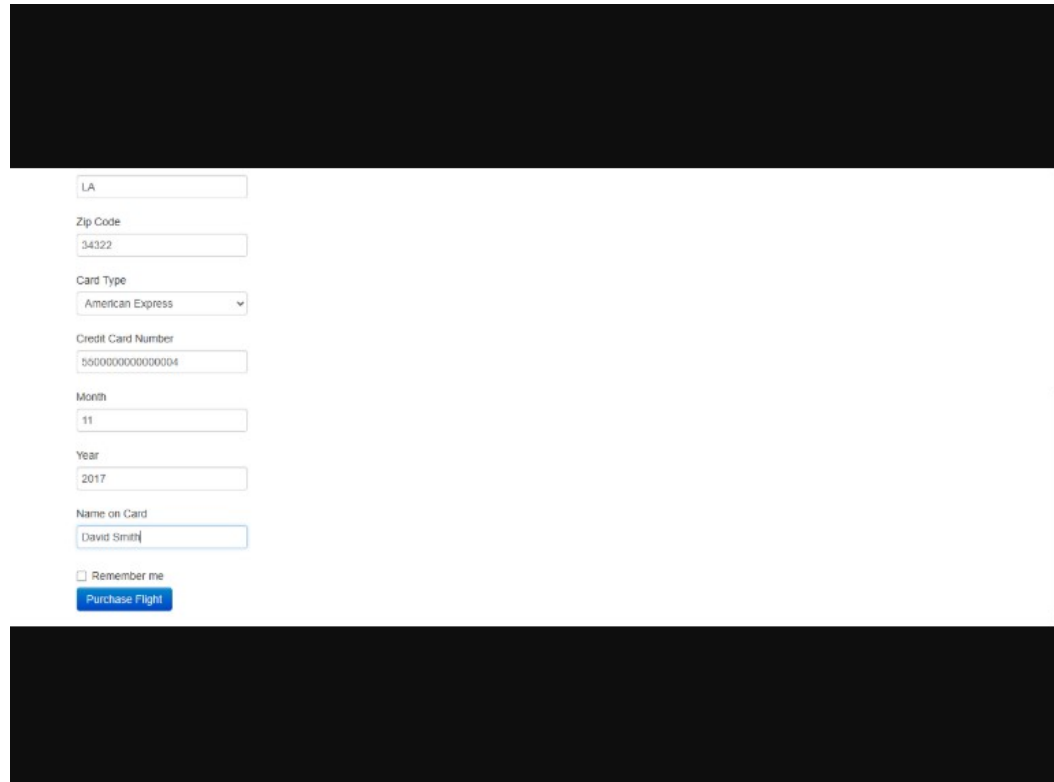
@DataProvider(name="userData")
public Object[][] getData(){
    return new Object[][] {
        {
            "John", "123 Main St", "New York", "NY", "12345", "Visa", "4111111111111111", "John Doe"
        },
        {
            "David", "Elm St", "Los Angeles", "LA", "34322", "American Express", "5500000000000004", "David Smith"
        }
    };
}

```

This allows the same test to run with different passenger details.

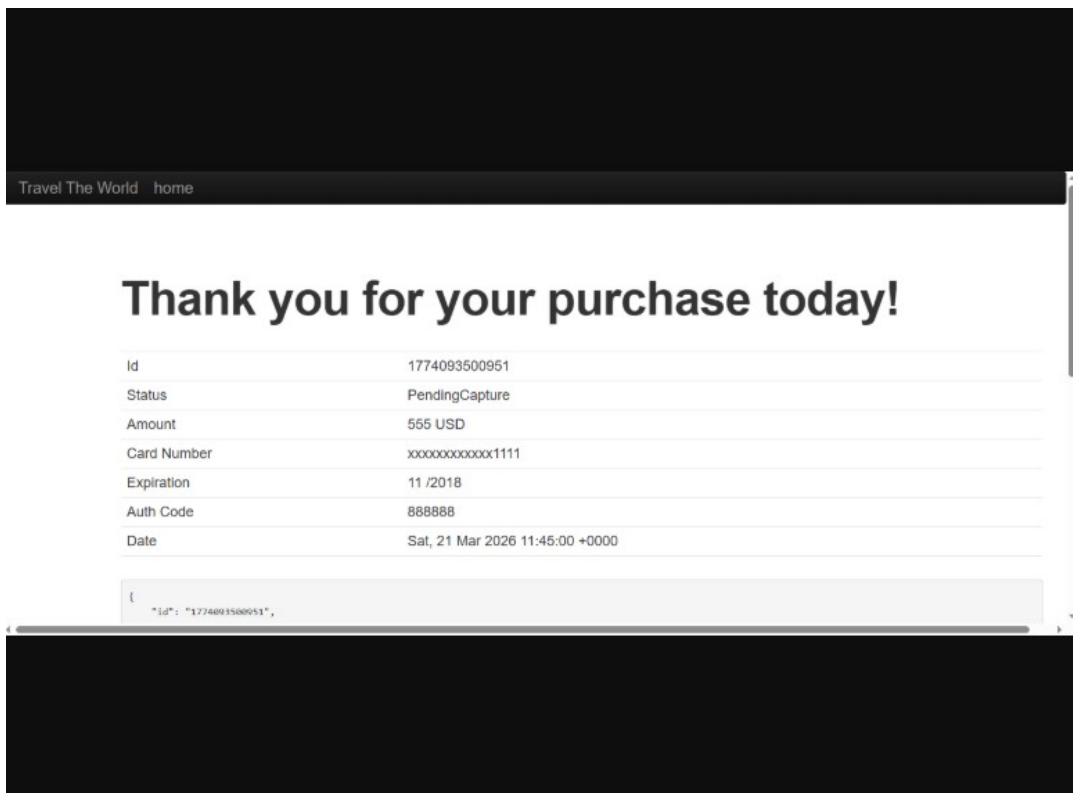
9. Screenshot Utility Implementation

In this automation framework, a reusable screenshot utility method has been implemented to capture screenshots during test execution.



The screenshot shows a flight booking form with the following fields and values:

- Location: LA
- Zip Code: 34322
- Card Type: American Express (dropdown menu)
- Credit Card Number: 5500000000000004
- Month: 11
- Year: 2017
- Name on Card: David Smith
- Remember me: ☐
- Purchase Flight: Purchase Flight



10. Test Execution Process

The test cases in this automation framework are executed using the **TestNG testing framework** integrated with **Selenium WebDriver**.

The execution process validates the functionality of the **BlazeDemo flight booking application** by automating key user actions such as searching flights, selecting flights, and completing the booking process.

11. Console Execution Screenshot

```
[RemoteTestNG] detected TestNG version 7.12.0
SLF4J(W): No SLF4J providers were found.
SLF4J(W): Defaulting to no-operation (NOP) logger implementation
SLF4J(W): See https://www.slf4j.org/codes.html#noProviders for further details.
Mar 21, 2026 5:14:34 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 146, returning the closest version; found: 145; Please upd
Starting the Flight Booking Test...
Test executing for user : John
-----
Application URL is correct! : https://blazedemo.com/
Total number of Departure Cities are : 7
Paris
Philadelphia
Boston
-----
Total number of Destination Cities are : 7
Buenos Aires
Rome
London
Berlin
New York
-----
Current URL is : https://blazedemo.com/reserve.php
Successfully navigated to Select Flight Page.
-----
Current URL is : https://blazedemo.com/purchase.php
Successfully navigated to Flight Purchase Page!
-----
UserName is : John
-----
Current URL is : https://blazedemo.com/confirmation.php
Navigate to confirmation page Successfully!
Confirmation Message is : Thank you for your purchase today!
Flight Booking Test Passed Successfully!
-----
Mar 21, 2026 5:14:47 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 146, returning the closest version; found: 145; Please upd
Starting the Flight Booking Test...
Test executing for user : David
-----
Application URL is correct! : https://blazedemo.com/
Total number of Departure Cities are : 7
Paris
Philadelphia
```

Mar 21, 2026 5:14:47 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 146, returning the closest version; found: 145; Plea:
Starting the Flight Booking Test...
Test executing for user : David

Application URL is correct! : https://blazedemo.com/
Total number of Departure Cities are : 7
Paris
Philadelphia
Boston

Total number of Destination Cities are : 7
Buenos Aires
Rome
London
Berlin
New York

Current URL is : https://blazedemo.com/reserve.php
Successfully navigated to Select Flight Page.

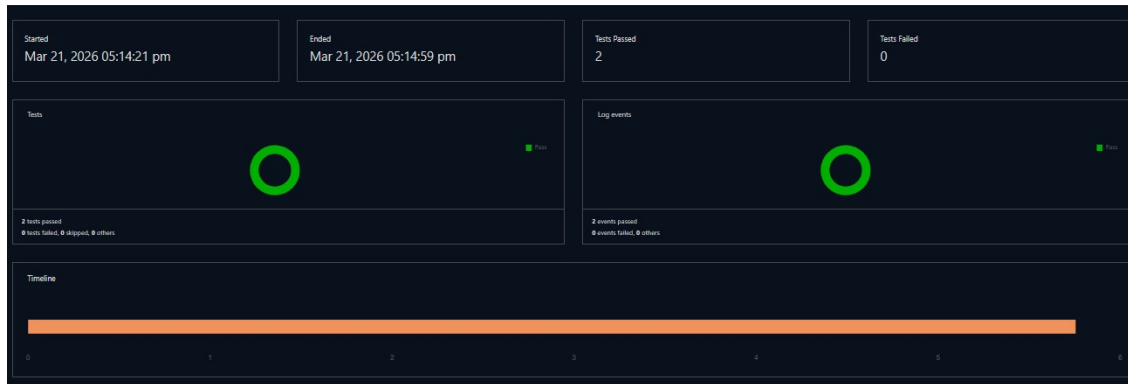
Current URL is : https://blazedemo.com/purchase.php
Successfully navigated to Flight Purchase Page!

UserName is : David

Current URL is : https://blazedemo.com/confirmation.php
Navigate to confirmation page Successfully!
Confirmation Message is : Thank you for your purchase today!
Flight Booking Test Passed Successfully!

=====
Suite
Total tests run: 2, Passes: 2, Failures: 0, Skips: 0
=====

12. Extent Report ScreenShot



Tests		testBookingflight	
testBookingflight	5:14:37 pm / 00:00:07:022	03.21.2026 5:14:37 pm	03.21.2026 5:14:44 pm 00:00:07:022 #test-id=1
testBookingflight		5:14:53 pm / 00:00:05:757	
		Thank you for your purchase today!	
STATUS		TIMESTAMP	
testBookingflightGot Successfully Pass!		5:14:43 pm	

13. Challenges Faced

Designing a proper automation framework structure using Page Object Model (POM). Implementing Page Factory to initialize web elements using Selenium WebDriver. Managing multiple test data sets using Data Driven Testing with TestNG.

Since **BlazeDemo** is a demo application, it has certain limitations. The application does not support negative scenarios, such as proper validation for invalid inputs and error handling. Therefore, automation testing was mainly focused on positive functional scenarios.

14. Test Scripts Screenshots

Com.blazedemo.pages

```
package com.blazedemo.pages;

import java.util.List;

public class HomePage {
    private WebDriver driver;

    //Locators
    //Departure City Dropdown
    @FindBy(name="fromPort")
    private WebElement depCity;

    // Destination City DropDown
    @FindBy(name="toPort")
    private WebElement desCity;

    // Find Flight Button
    @FindBy(xpath="//input[@value=\"Find Flights\"]")
    private WebElement findFlights;

    // Constructor
    public HomePage(WebDriver driver) {
        this.driver=driver;
        PageFactory.initElements(driver,this);
    }

    // Methods
    // Method to get the current URL of the application
    public String getAppURL() {
        return driver.getCurrentUrl();
    }

    // Method to get Departure City
    public void chooseDepartureCity(String fromCity) {
        Select select = new Select(depCity);
        List<WebElement>alldepCities = select.getOptions();
        System.out.println("Total number of Departure Cities are : "+alldepCities.size());

        for(WebElement city:alldepCities) {
            System.out.println((city.getText()));
            if(city.getText().contains(fromCity)){
                city.click();
                break;
            }
        }
    }
}
```

```

    }
    System.out.println("-----");
}

// Method to get Destination City
public void chooseDestinationCity(String toCity) {
    Select select = new Select(desCity);
    List<WebElement>alldesCities=select.getOptions();
    System.out.println("Total number of Destination Cities are : "+alldesCities.size());

    for(WebElement city:alldesCities) {
        System.out.println((city.getText()));
        if(city.getText().contains(toCity)){
            city.click();
            break;
        }
    }
    System.out.println("-----");
}

// Method to click on Find Flights Button
public void clickFindFlights() {
    findFlights.click();
}

// Method to verify navigation to Select Flight Page
public void verifyNavigateToSelectFlightPage() {
    String currentUrl = driver.getCurrentUrl();
    System.out.println("Current URL is : " + currentUrl);
    Assert.assertTrue(currentUrl.contains("reserve"), "Navigation to Select Flight Page failed.");
    System.out.println("Successfully navigated to Select Flight Page.");
    System.out.println("-----");
}
}
}

```

```

package com.blazedemo.pages;

import org.openqa.selenium.WebDriver;

public class ReservePage {
    WebDriver driver;

    // Locator for Choose Flight Button
    @FindBy(xpath="//input[@class=\"btn btn-small\"])[2]")
    private WebElement chooseFlight;

    // Constructor
    public ReservePage(WebDriver driver) {
        this.driver=driver;
        PageFactory.initElements(driver,this);
    }

    // Method to click on Choose Flight Button
    public void selectFlight() {
        chooseFlight.click();
    }

    // Method to verify navigation to Purchase Page
    public void verifyNavigationToPurchasePage() {
        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
        String partialUrl = driver.getCurrentUrl();
        System.out.println("Current URL is : " + partialUrl);

        Assert.assertTrue(partialUrl.contains("purchase"),"Navigate to Flight Purchase Page Failed!");
        System.out.println("Successfully navigated to Flight Purchase Page!");
        System.out.println("-----");
    }
}

```

```

package com.blazedemo.pages;

import org.openqa.selenium.By;

public class PurchasePage {
    WebDriver driver;

    @FindBy(id="inputName") private WebElement name;
    @FindBy(id="address") private WebElement address;
    @FindBy(id="city") private WebElement city;
    @FindBy(id="state") private WebElement state;
    @FindBy(id="zipCode") private WebElement zipCode;
    @FindBy(id="cardType") private WebElement cardType;
    @FindBy(id="creditCardNumber") private WebElement CCNumber;
    @FindBy(id="nameOnCard") private WebElement nameOnCard;
    @FindBy(id="rememberMe") private WebElement rememberMe;
    @FindBy(xpath="//input[@class=\"btn btn-primary\"]") private WebElement purchaseFlight;

    // Constructor
    public PurchasePage(WebDriver driver) {
        this.driver=driver;
        PageFactory.initElements(driver,this);
    }

    // Method to fill the form and purchase the flight
    public void enterPassengerDetails(String uname, String uaddress, String ucity, String ustate, String uzipCode, String ucardType, String uCCNumber,String unameOnCard) {
        name.sendKeys(uname);
        address.sendKeys(uaddress);
        city.sendKeys(ucity);
        state.sendKeys(ustate);
        zipCode.sendKeys(uzipCode);
        Select Ctype = new Select(cardType);
        Ctype.selectByVisibleText(ucardType);
        CCNumber.sendKeys(uCCNumber);
        //Month.sendKeys(month);
        //Year.sendKeys(year);
        nameOnCard.sendKeys(unameOnCard);
    }

    // Method to click on Purchase Flight Button
    public void clickPurchaseFlight() {
        rememberMe.click();
        purchaseFlight.click();

        String nextPageUrl = driver.getCurrentUrl();
        System.out.println("Current URL is : " + nextPageUrl);
        Assert.assertTrue(nextPageUrl.contains("confirmation"),"Purchase Flight Failed!");
        System.out.println("Navigate to confirmation page Successfully!");
        //System.out.println("Successful message text is :"+driver.findElement(By.tagName("h1")).getText());
    }
}

```

```

package com.blazedemo.pages;

import org.openqa.selenium.WebDriver;

public class ConfirmationPage {
    WebDriver driver;

    @FindBy(xpath="//h1")
    private WebElement ConfirmationMsg;

    // Constructor
    public ConfirmationPage(WebDriver driver) {
        this.driver=driver;
        PageFactory.initElements(driver,this);
    }

    // Method to get the Confirmation Message
    public String getConfirmationMessage() {
        return ConfirmationMsg.getText();
    }
}

```

com.blazedemo.utils

```

package com.blazedemo.utils;

import org.testng.annotations.DataProvider;

public class DataProviderClass {

    @DataProvider(name="userData")
    public Object[][] getData(){
        return new Object[][] {
            {
                "John",
                "123 Main St",
                "New York",
                "NY",
                "12345",
                "Visa",
                "4111111111111111",
                "John Doe"
            },
            {
                "David",
                "Elm St",
                "Los Angeles",
                "LA",
                "34322",
                "American Express",
                "5500000000000004",
                "David Smith"
            }
        };
    }
}

```

```

package com.blazedemo.utils;

import java.io.File;

public class ScreenshotUtil {

    public static String getScreenshot(WebDriver driver, String name) {
        TakesScreenshot ts = (TakesScreenshot) driver;

        File src= ts.getScreenshotAs(OutputType.FILE);
        String folderpath = System.getProperty("user.dir") + "/BlazeDemoScreenshots/";
        File folder= new File(folderpath);

        if(!folder.exists()) {
            folder.mkdirs();
        }

        String path = folderpath + name + System.currentTimeMillis() + ".png";
        File dest = new File(path);

        try {
            FileHandler.copy(src, dest);
        } catch (IOException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }

        return path;
    }
}

```

Com.testlisteners

```

package com.testlisteners;

import org.testng.ITestContext;

public class ExtentTestNGListeners implements ITestListener {
    /*
     * ThreadLocal is a class in java.lang package provides a way to store values in per thread basis
     * It create separate instance of the value for each thread allowing each thread to have its own copy of the value
     * without interfering with other threads.
     *
     * It is useful for storing values that are specific to a single thread such as transaction or request specific data
     * ThreadLocal provides a simple and efficient way to store thread local variables without having to pass them as method arguments or store them as instance variables.
     */
    ExtentReports extent=ExtentManager.getInstance();
    ThreadLocal<ExtentTest> parentTest=new ThreadLocal<ExtentTest>();

    public void onStart(ITestResult result)
    {
        ExtentTest extentTest=extent.createTest(result.getMethod().getMethodName());
        parentTest.set(extentTest);
    }

    public void onSuccess(ITestResult result) {
        String path =ScreenshotUtil.getScreenshot(BaseTest.driver, result.getMethod().getMethodName());
        //parentTest.get().pass("Test Passed");
        parentTest.get().log(Status.PASS,result.getName()+"Got Successfully Pass!").addScreenCaptureFromPath(path);
    }

    public void onFailure(ITestResult result)
    {
        parentTest.get().fail("Test Failed "+result.getThrowable().getMessage());
        System.out.println("Test Failure : "+result.getMethod().getMethodName());
        parentTest.get().log(Status.FAIL,result.getName()+"Test Fail");
    }

    public void onTestSkipped(ITestResult result) {
        parentTest.get().skip("Test Skipped "+result.getThrowable().getMessage());
    }

    public void onFinish(ITestContext context) {
        extent.flush();
    }
}

```

```

package com.testlisteners;

import java.util.Date;

public class ExtentManager {
    public static ExtentReports extent;

    public static ExtentReports getInstance()
    {
        if(extent==null)
        {
            extent=createInstance();
            return extent;
        }
        else
        {
            return extent;
        }
    }

    static String rname;
    public static ExtentReports createInstance()
    {
        String timestamp=new SimpleDateFormat("yyyy.MM.dd.HH.mm.ss").format(new Date());
        rname="BlazeDemoAutomationTestReport"+timestamp+".html";

        ExtentSparkReporter sparkReporter=new ExtentSparkReporter(System.getProperty("user.dir")+"/TestReports/"+rname);
        sparkReporter.config().setTheme(Theme.DARK);
        sparkReporter.config().setReportName("BlazeDemoAutomationTestReport");
        sparkReporter.config().setDocumentTitle("BlazeDemoAutomation_Report");

        ExtentReports extent=new ExtentReports();
        extent.attachReporter(sparkReporter);

        return extent;
    }
}

```


Com.blazedemo.tests

```
package com.blazedemo.tests;

import java.time.Duration;

public class BaseTest {
    public static WebDriver driver;

    @BeforeMethod
    public void setup() {

        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.get("https://blazedemo.com/");

    }

    @AfterMethod
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}

package com.blazedemo.tests;

import org.testng.Assert;

Run ALL
public class FlightBookingTest extends BaseTest {

    // This test method is designed to book a flight on the Blazedemo website.
    // Using DataProvider to run the test with multiple sets of data

    @Test(dataProvider="userData",dataProviderClass=com.blazedemo.utils.DataProviderClass.class)
    Run | Debug
    public void testBookingflight(String name,String address,String city,String state,String zipCode,String cardType,String CCNumber,String nameOnCard)
    {
        System.out.println("Starting the Flight Booking Test...");
        // Print the test data for the current iteration
        System.out.println("Test executing for user : " + name);
        System.out.println("-----");
        // Create an object of HomePage and perform actions
        HomePage home = new HomePage(driver);

        // Verify the application URL
        String actUrl = home.getAppURL();
        Assert.assertTrue(actUrl.contains("blazedemo"), "Application URL is incorrect!");
        System.out.println("Application URL is correct! : " + actUrl);

        // Select Departure and Destination Cities and click on Find Flights
        home.chooseDepartureCity("Boston");
        home.chooseDestinationCity("New York");
        home.clickFindFlights();
        home.verifyNavigateToSelectFlightPage();

        // Create an object of ReservePage and perform actions
        ReservePage reserve = new ReservePage(driver);
        reserve.selectFlight();
        reserve.verifyNavigationToPurchasePage();

        // Print the test data for the current iteration
        System.out.println("UserName is : " + name);
        System.out.println("-----");

        // Create an object of PurchasePage and perform actions
        PurchasePage purchase = new PurchasePage(driver);
        purchase.enterPassengerDetails(name, address, city, state, zipCode, cardType, CCNumber, nameOnCard);
    }
}
```



```

        //capture screenshot of the Purchase page after filling details and before clicking on Purchase Flight
        ScreenshotUtil.getScreenshot(driver, "PurchasePage_AfterClickingPurchaseFlight");

        purchase.clickPurchaseFlight();

        // create an object of ConfirmationPage and perform actions
        ConfirmationPage confirm = new ConfirmationPage(driver);
        String Message = confirm.getConfirmationMessage();
        System.out.println("Confirmation Message is : " + Message);
        Assert.assertTrue(Message.contains("Thank you"), " Flight Booking Failed!");
        System.out.println("Flight Booking Test Passed Successfully!");

        // Capture screenshot of the Confirmation page
        ScreenshotUtil.getScreenshot(driver, "ConfirmationPage");
        System.out.println("-----");
    }
}

```

Testng.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Suite">
    <listeners>
        <listener class-name="com.testlisteners.ExtentITestNGListeners"> </listener>
    </listeners>
    <test thread-count="5" name="Test">
        <classes>
            <class name="com.blazedemo.tests.FlightBookingTest"/>
        </classes>
    </test> <!-- Test -->
</suite> <!-- Suite -->

```

15.Conclusion

This project demonstrates the successful automation of the **BlazeDemo flight booking application** using **Selenium WebDriver** with **Java** and the **TestNG** framework.